

claude-windows / 54d4074a-5a8d-4058-b42d-a757c338132c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra\54d4074a-5a8d-4058-b42d-a757c338132c\subagents\workflows\wf_1c9ea2fd-eb8\agent-a462b4edee9472ba1.jsonl`  
- SHA-256: `6a63658acb04188dfd98c5a4a9b225fd9828908eef553e49a865bd8d08d0b642`  
- Source modified: `2026-06-21T19:33:33+00:00`  
- Imported at: `2026-07-05T16:48:22+00:00`  
- project: `wf_1c9ea2fd-eb8`  
- session_id: `54d4074a-5a8d-4058-b42d-a757c338132c`
```

Transcript

1. user (2026-06-21T19:32:10.684Z)

You are reviewing a PR that stands up the FIRST coverage floor for the web SPA. Read these files under C:/Users/avidu/Projects/khelsutra-cov:

- web/vite.config.ts (the new coverage block + thresholds)
- web/package.json

Adversarially assess the COVERAGE CONFIG ONLY:

1. Are the excludes justified? Specifically main.tsx (entry bootstrap), src/api/types.ts (type-only), src/test/** (jsdom shim), **/*.d.ts ? does any exclude hide REAL runtime logic that should be covered? Open each excluded file and check it truly has no testable logic.
 2. Is `all: true` + `include: ["src/\*\*/\*.{ts,tsx}"]` correct so untested new files DROP coverage (not silently ignored)?
 3. Are the thresholds (statements 92, branches 87, functions 80, lines 92) actually BELOW current measured coverage (92.61/87.74/82.35/92.61) so CI is green, yet tight enough to catch a real regression? Flag if any threshold is above current (would fail CI) or so loose it is meaningless.
 4. Could the gate be trivially gamed?
- Return structured findings. Do NOT run long test/coverage commands (already verified); reason statically by reading files.

2. assistant (2026-06-21T19:32:13.093Z)

I'll review the coverage config. Let me read the relevant files first.

3. user (2026-06-21T19:32:14.272Z)

```
1      import { defineConfig } from "vitest/config";  
2      import react from "@vitejs/plugin-react";  
3      import tailwindcss from "@tailwindcss/vite";  
4  
5      // The SPA makes SAME-ORIGIN relative `/api/*` calls. In the appliance (prod) a reverse  
6      // proxy serves the SPA and proxies `/api` to the engine on 127.0.0.1:8000  
7      // (docs/LOCAL_APPLIANCE_ARCHITECTURE.md). In dev, Vite proxies `/api` to the local  
engine  
8      // (`python -m backend.main --server`, :8000). Override with VITE_ENGINE_ORIGIN if  
needed.  
9      export default defineConfig({  
10         plugins: [react(), tailwindcss()],  
11         server: {  
12             proxy: {  
13                 "/api": { target: "http://localhost:8000", changeOrigin: true },  
14             },  
15         },  
16         test: {  
17             // Default env is node (the pure lib/parser tests); the i18n render test opts into  
jsdom via a
```

```

18      // `/// @vitest-environment jsdom` docblock. Match .test.tsx too so render tests are
discovered.
19      environment: "node",
20      include: ["src/**/*.test.{ts,tsx}"],
21      // DOM-env localStorage polyfill (jsdom under Node 26 ships none); node-env files
no-op.
22      setupFiles: ["src/test/setup.ts"],
23      // Coverage floor (CI runs `npm run test:coverage`): a merge can't DROP coverage
below these
24      // thresholds, so untested new code is caught. `all` counts every source file (not
only the ones a
25      // test imported), so deleting a test surfaces here too. Excludes are non-logic
files: the app
26      // entry/bootstrap, type-only declarations, and the test harness itself.
27      coverage: {
28        provider: "v8",
29        reporter: ["text", "text-summary"],
30        all: true,
31        include: ["src/**/*.test.{ts,tsx}"],
32        exclude: [
33          "src/**/*.test.{ts,tsx}", // the tests
34          "src/test/**", // jsdom localStorage shim (test harness, not shipped logic)
35          "src/main.tsx", // app entry: mounts <App/> + bundles fonts ? nothing to unit-
test
36          "src/api/types.ts", // type-only declarations (no runtime code)
37          "**/*.d.ts",
38        ],
39        // Floor set just below current (stmts/lines 92.6, branches 87.7, funcs 82.4) ? a
real ratchet
40        // that fails CI on a meaningful drop, with a little slack so it doesn't flap.
Ratchet upward as
41        // coverage grows. (functions is kept looser: App.tsx is the engine-orchestrating
shell, costly
42        // to exercise fully in unit tests.)
43        thresholds: {
44          statements: 92,
45          branches: 87,
46          functions: 80,
47          lines: 92,
48        },
49      },
50    },
51  });
52

```

4. user (2026-06-21T19:32:14.294Z)

```

1    {
2      "name": "khelsutra-web",
3      "private": true,
4      "version": "0.0.0",
5      "type": "module",
6      "description": "KhelSutra Studio ? operator console SPA (drives the rally indexer
engine).",
7      "scripts": {
8        "dev": "vite",
9        "build": "vite build",
10       "preview": "vite preview",
11       "typecheck": "tsc --noEmit",
12       "test": "vitest run",
13       "test:coverage": "vitest run --coverage"
14     },
15     "dependencies": {
16       "@fontsource-variable/noto-sans-devanagari": "^5.2.8",
17       "@fontsource-variable/noto-sans-kannada": "^5.2.9",

```

```

18     "i18next": "^26.3.1",
19     "i18next-browser-languagedetector": "^8.2.1",
20     "i18next-icu": "^2.4.3",
21     "react": "^19.0.0",
22     "react-dom": "^19.0.0",
23     "react-i18next": "^17.0.8"
24   },
25   "devDependencies": {
26     "@tailwindcss/vite": "^4.0.0",
27     "@testing-library/dom": "^10.4.1",
28     "@testing-library/react": "^16.3.2",
29     "@types/node": "^26.0.0",
30     "@types/react": "^19.0.0",
31     "@types/react-dom": "^19.0.0",
32     "@vitejs/plugin-react": "^4.3.4",
33     "@vitest/coverage-v8": "^3.2.6",
34     "jsdom": "^29.1.1",
35     "tailwindcss": "^4.0.0",
36     "typescript": "^5.7.0",
37     "vite": "^6.0.0",
38     "vitest": "^3.0.0"
39   }
40 }
41

```

5. assistant (2026-06-21T19:32:17.358Z)

Now let me examine the excluded files to verify they truly have no testable runtime logic. Let me find and read each one.

6. user (2026-06-21T19:32:18.295Z)

```

1   import { StrictMode } from "react";
2   import { createRoot } from "react-dom/client";
3   import App from "./App";
4   import "./i18n"; // initialize the i18next singleton before the app renders (tracker B1)
5   // Self-hosted OFL Noto fonts so Kannada/Hindi render crisply offline (tracker B4 /
decision D7) ?
6   // no Google Fonts runtime dependency. unicode-range-scoped, so only Indic codepoints
use them.
7   import "@fontsource-variable/noto-sans-kannada/index.css";
8   import "@fontsource-variable/noto-sans-devanagari/index.css";
9   import "./index.css";
10
11   const root = document.getElementById("root");
12   if (!root) throw new Error("#root not found");
13
14   createRoot(root).render(
15     <StrictMode>
16       <App />
17     </StrictMode>,
18   );
19

```

7. user (2026-06-21T19:32:18.441Z)

```

1   // Types mirror the engine's verified contract (badminton-highlight-indexer).
2   // Anchors: backend/infrastructure/database.py:124-135 (play_segments),
3   // backend/main.py:331-396 (query/compile), :307-328 (highlights), :121 (config).
4
5   export interface RallyMetadata {
6     /** Gemini's shuttle_exchange_count (fallback max(3, dur/0.8)); absent on some
detectors. */
7     shot_count?: number;

```

```

8      /** Gemini confidence STRING (often "Unknown") ? not a numeric meter. */
9      shot_count_confidence?: string;
10     detector?: string;
11     [key: string]: unknown;
12 }
13
14 export interface RallySegment {
15     /** The integer the UI selects and passes to /api/compile as segment_ids. */
16     id: number;
17     start_time: number;
18     end_time: number;
19     duration: number;
20     /** motion-segmenter FABRICATION (random) ? DO NOT surface (see
PER_RALLY_SELECTION.md). */
21     server: string | null;
22     /** motion-segmenter FABRICATION (random) ? DO NOT surface. */
23     receiver: string | null;
24     metadata: RallyMetadata;
25     /** motion-only fabricated sub-events ? DO NOT surface. */
26     events?: unknown[];
27 }
28
29 export interface QueryResponse {
30     play_segments: RallySegment[];
31 }
32
33 export interface CompileResponse {
34     status: string;
35     /** server-local path (NOT a URL) ? build the download URL with highlightUrl(). */
36     filepath: string;
37 }
38
39 export interface EngineConfig {
40     stitching?: {
41         /** The shot-count floor /api/compile silently enforces ? surface it, don't hide it.
*/
42         min_shot_count?: number;
43         [key: string]: unknown;
44     };
45     [key: string]: unknown;
46 }
47
48 /** POST /api/process ? 202 [backend/main.py:260]. The job is queued; poll GET
/api/jobs/{job_id}. */
49 export interface ProcessResponse {
50     job_id: string;
51     /** Always "queued" at submit. */
52     status: string;
53     /** Relative poll path, e.g. "/api/jobs/<id>". */
54     poll: string;
55 }
56
57 /** Chunked upload (backend/api/uploads.py). The browser uploads a local file in
sequential parts
58 * ? max_part_mb, then `complete` returns a server-host `path` to feed straight into
POST /api/process. */
59 export interface UploadInitResponse {
60     upload_id: string;
61     /** Per-part ceiling in MB ? slice the file into parts this big. */
62     max_part_mb: number;
63     next_part: string;
64 }
65 export interface UploadPartResponse {
66     upload_id: string;
67     received_parts: number;

```

```

68     received_bytes: number;
69 }
70 export interface UploadCompleteResponse {
71     /** Absolute path on the engine host ? pass as video_path to POST /api/process. */
72     path: string;
73     video_id: string;
74     size_bytes: number;
75 }
76
77 /** GET /api/jobs/{job_id} [backend/main.py:303]. Two status layers:
78  * `status` = the EXECUTION state; `result.status` = the pipeline VERDICT
79  * A `done` job whose `result.status` === "failed" (or with no video_id) is a success-
80  * shaped
81  * FAILURE ? the UI must not treat it as a load. */
82 export interface JobStatus {
83     job_id: string;
84     status: "queued" | "running" | "done" | "failed";
85     /** Free-text coarse phase ("hashing", "segmenting (gemini)", "finished") ? NOT a
86     percentage. */
87     progress?: string | null;
88     /** The canonical video_id, surfaced top-level on success (also in result.video_id).
89     */
90     video_id?: string | null;
91     /** Execution error (worker crash / unexpected) when status === "failed". */
92     error?: string | null;
93     result?: { status?: string; video_id?: string; message?: string } | null;
94 }

```

8. assistant (2026-06-21T19:32:21.184Z)

Both confirmed as non-logic. Now let me look at the test harness directory and the full source tree to verify `include` matches real files and there are no other source files being silently excluded.

9. user (2026-06-21T19:32:22.093Z)

No files found

10. user (2026-06-21T19:32:24.085Z)

```

C:/Users/avidu/Projects/khelsutra-cov/web/src/App.tsx
C:/Users/avidu/Projects/khelsutra-cov/web/src/api/client.test.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/api/client.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/api/locale.test.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/api/types.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/api/upload.test.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/components/CompileResult.tsx
C:/Users/avidu/Projects/khelsutra-cov/web/src/components/IngestPanel.test.tsx
C:/Users/avidu/Projects/khelsutra-cov/web/src/components/IngestPanel.tsx
C:/Users/avidu/Projects/khelsutra-cov/web/src/components/LanguageSwitcher.tsx
C:/Users/avidu/Projects/khelsutra-cov/web/src/components/QueryBar.examples.test.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/components/QueryBar.tsx
C:/Users/avidu/Projects/khelsutra-cov/web/src/components/RallyGrid.tsx
C:/Users/avidu/Projects/khelsutra-cov/web/src/components/SelectionFooter.tsx
C:/Users/avidu/Projects/khelsutra-cov/web/src/components/StatusChip.tsx
C:/Users/avidu/Projects/khelsutra-cov/web/src/components/TabFilters.test.tsx
C:/Users/avidu/Projects/khelsutra-cov/web/src/components/TabFilters.tsx
C:/Users/avidu/Projects/khelsutra-cov/web/src/components/a4.touch.test.tsx
C:/Users/avidu/Projects/khelsutra-cov/web/src/components/a8.status.test.tsx
C:/Users/avidu/Projects/khelsutra-cov/web/src/components/build-error.test.tsx
C:/Users/avidu/Projects/khelsutra-cov/web/src/components/upload.test.tsx
C:/Users/avidu/Projects/khelsutra-cov/web/src/hooks/useIngestJob.test.ts

```

```

C:/Users/avidu/Projects/khelsutra-cov/web/src/hooks/useIngestJob.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/hooks/useRallySelection.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/hooks/useUpload.test.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/hooks/useUpload.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/i18n/hygiene.test.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/i18n/i18n.test.tsx
C:/Users/avidu/Projects/khelsutra-cov/web/src/i18n/index.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/index.css
C:/Users/avidu/Projects/khelsutra-cov/web/src/lib/errorText.test.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/lib/errorText.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/lib/errors.test.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/lib/errors.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/lib/floor.test.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/lib/floor.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/lib/format.test.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/lib/format.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/lib/log.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/lib/query.test.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/lib/query.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/lib/selection.test.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/lib/selection.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/main.tsx
C:/Users/avidu/Projects/khelsutra-cov/web/src/test/setup.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/vite-env.d.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/i18n/catalog-invariants.test.ts
C:/Users/avidu/Projects/khelsutra-cov/web/src/components/SelectionFooter.test.tsx

```

11. assistant (2026-06-21T19:32:28.970Z)

Now I have the full picture. Let me read the test harness setup file, and the source files that have NO corresponding test file to check whether untested logic exists that the gate would (correctly) catch or that might be hidden. Key files without a co-located test: `App.tsx`, several components (`CompileResult.tsx`, `LanguageSwitcher.tsx`, `RallyGrid.tsx`, `StatusChip.tsx`), `hooks/useRallySelection.ts`, `lib/log.ts`, `i18n/index.ts`. Let me check the test setup and `lib/log.ts` (a potential gaming vector) and a couple of others.

12. user (2026-06-21T19:32:29.455Z)

```

1      // Vitest setup (runs before each test file's imports). DOM-env only: jsdom under Node
26 does not
2      // expose a Web Storage `localStorage`, which the i18n language detector and the
persistence test
3      // both need. Node-env test files (typeof window === "undefined") skip this entirely.
4      if (typeof window !== "undefined" && typeof globalThis.localStorage === "undefined") {
5          let store: Record<string, string> = {};
6          const mem: Storage = {
7              get length() {
8                  return Object.keys(store).length;
9              },
10             clear() {
11                 store = {};
12             },
13             getItem(key: string) {
14                 return Object.prototype.hasOwnProperty.call(store, key) ? store[key] : null;
15             },
16             key(index: number) {
17                 return Object.keys(store)[index] ?? null;
18             },
19             removeItem(key: string) {
20                 delete store[key];
21             },
22             setItem(key: string, value: string) {
23                 store[key] = String(value);
24             },
25         };

```

```

26     Object.defineProperty(window, "localStorage", { value: mem, configurable: true,
writable: true });
27     Object.defineProperty(globalThis, "localStorage", { value: mem, configurable: true,
writable: true });
28   }
29

```

13. user (2026-06-21T19:32:29.941Z)

```

1     /// <reference types="vite/client" />
2
3     // Augment Vite's env typing with the SPA's build-time vars (declaration merging with
vite/client's
4     // ImportMetaEnv). See web/src/api/client.ts::apiBase.
5     interface ImportMetaEnv {
6       /** Engine API base for a SPLIT deploy (SPA on Cloudflare Pages + engine on a separate
`api.*`
7       * cloudflared tunnel ? HOSTING_PLAN.md §2). Include the `/api` path, e.g.
8       * `https://api.khelsutra.guru/api`. Unset ? same-origin `"/api"` (dev proxy /
single-origin appliance). */
9       readonly VITE_API_BASE?: string;
10    }
11

```

14. user (2026-06-21T19:32:30.401Z)

```

1     // Structured client logger ? the observability seed for the operator console
2     // (docs/LOCAL_APPLIANCE_ARCHITECTURE.md §5). Every API call carries a CORRELATION ID so
a
3     // single user journey is traceable across the UI ? FastAPI ? worker. Logs are JSON-
shaped
4     // so they aggregate cleanly off a transient host. Keep this dependency-free.
5
6     export type LogLevel = "debug" | "info" | "warn" | "error";
7
8     /** A short, unique id to correlate all log lines (and the x-correlation-id header) for
one action. */
9     export function newCorrelationId(): string {
10       const c = (globalThis as { crypto?: Crypto }).crypto;
11       if (c && typeof c.randomUUID === "function") return c.randomUUID();
12       // Fallback for environments without crypto.randomUUID.
13       return `cid_${Date.now().toString(36)}_${Math.floor(Math.random() *
1e9).toString(36)}`;
14     }
15
16     export function log(level: LogLevel, event: string, fields: Record<string, unknown> =
{}): void {
17       const line = JSON.stringify({ ts: new Date().toISOString(), level, event, ...fields
});
18       if (level === "error") console.error(line);
19       else if (level === "warn") console.warn(line);
20       else console.log(line);
21     }
22

```

15. assistant (2026-06-21T19:32:36.052Z)

`vite-env.d.ts` is covered by the `\*\*/\*.d.ts` exclude (correct ? type augmentation only).
`src/test/setup.ts` is real runtime code but is genuinely the test harness ? correctly excluded.

`lib/log.ts` is interesting: it has real runtime logic (`newCorrelationId` with a branch +
fallback, `log` with 3 branches) and is NOT excluded, so it counts toward coverage. That's
correct behavior ? it's included and must be tested or it drags coverage down. Let me verify the
`all: true` behavior interaction and confirm there's no separate vitest config or coverage

override in package.json or elsewhere that would change the picture. Let me also confirm whether `lib/log.ts` has a test (it has no co-located `.test`).

16. user (2026-06-21T19:32:38.701Z)

Found 7 files

```
C:\Users\avidu\Projects\kheIsutra-cov\web\src\lib\log.ts
C:\Users\avidu\Projects\kheIsutra-cov\web\src\i18n\hygiene.test.ts
C:\Users\avidu\Projects\kheIsutra-cov\web\src\hooks\useUpload.ts
C:\Users\avidu\Projects\kheIsutra-cov\web\src\hooks\useIngestJob.ts
C:\Users\avidu\Projects\kheIsutra-cov\web\src\components\IngestPanel.test.tsx
C:\Users\avidu\Projects\kheIsutra-cov\web\src\api\client.ts
C:\Users\avidu\Projects\kheIsutra-cov\web\src\App.tsx
```

17. user (2026-06-21T19:32:38.755Z)

No files found

18. user (2026-06-21T19:32:39.202Z)

No files found

19. assistant (2026-06-21T19:32:45.161Z)

No separate vitest config ? `vite.config.ts` is authoritative. `log.ts` is exercised transitively through `client.ts`/`useUpload.ts`/`useIngestJob.ts` tests, so it contributes covered lines rather than being a hole. Good.

Now let me verify the threshold semantics carefully. Vitest thresholds are a floor (coverage must be `>=` threshold). Current measured: statements 92.61, branches 87.74, functions 82.35, lines 92.61. Thresholds: 92 / 87 / 80 / 92.

Let me double check there's no `perFile` or `100`/auto-update threshold flag, and confirm the `include`/`exclude` glob interaction once more by checking if `i18n/index.ts` (a config-ish file but with runtime init logic) is included.

20. user (2026-06-21T19:32:45.636Z)

```
1      // i18n scaffold (tracker B1) ? the shared localization foundation for KheIsutra Studio.
2      //
3      // Per docs/FRIENDLY_AND_MULTILINGUAL.md + the engine's I18N_PLAN.md:
4      //   - keys, never prose (t("app.title"), enforced later by a no-bare-strings lint, B2);
5      //   - ICU MessageFormat for interpolation/plurals (Kannada/Hindi plural rules live in
the catalog,
6      //     not the code ? replaces the SPA's hand-rolled `count > 1 ? "s" : ""`);
7      //   - one negotiation order shared with the marketing site (B3);
8      //   - always-fallback to `en` so a missing translation degrades to English, never a raw
key.
9      import i18n from "i18next";
10     import { initReactI18next } from "react-i18next";
11     import LanguageDetector from "i18next-browser-languagedetector";
12     import ICU from "i18next-icu";
13     // Catalogs are HOISTED to repo-root /locales (D-CATALOG-HOME) ? the single source of
truth shared
14     // by this SPA and the marketing site's t() shim (tracker B3). Vite resolves the
workspace root via
15     // the repo `.git` marker, so importing the parent-dir /locales works in dev, build, and
vitest.
16     import enCommon from "../../locales/en/common.json";
17     import knCommon from "../../locales/kn/common.json";
18     import hiCommon from "../../locales/hi/common.json";
19     import esCommon from "../../locales/es/common.json";
20     import daCommon from "../../locales/da/common.json";
21     import idCommon from "../../locales/id/common.json";
```



```

22
23 // Supported locales: en/kn/hi foundation (FRIENDLY D6) + MULTILINGUAL_EXPANSION Phase A
(es/da/id,
24 // Latin, catalog-only). Anything else falls back to `en`.
25 export const SUPPORTED_LOCALES = ["en", "kn", "hi", "es", "da", "id"] as const;
26 export type Locale = (typeof SUPPORTED_LOCALES)[number];
27 export const DEFAULT_LOCALE: Locale = "en";
28
29 // ONE persisted-preference key, shared with the marketing site's t() shim (D9 / tracker
B3), so a
30 // visitor's language choice carries across both surfaces.
31 export const LANG_STORAGE_KEY = "khelsutra.lang";
32
33 // Catalogs live in repo-root /locales (D-CATALOG-HOME), consumed by both surfaces. `kn`
+ `hi`
34 // shipped (B5/B6, AI-drafted ? native review pending). A missing key in any locale
still falls back
35 // to `en` ? the by-design invariant.
36 const resources = {
37   en: { common: enCommon },
38   kn: { common: knCommon },
39   hi: { common: hiCommon },
40   es: { common: esCommon },
41   da: { common: daCommon },
42   id: { common: idCommon },
43 } as const;
44
45 void i18n
46   .use(ICU)
47   .use(LanguageDetector)
48   .use(initReactI18next)
49   .init({
50     resources,
51     fallbackLng: DEFAULT_LOCALE,
52     supportedLngs: [...SUPPORTED_LOCALES],
53     nonExplicitSupportedLngs: true, // en-IN -> en
54     defaultNS: "common",
55     ns: ["common"],
56     interpolation: { escapeValue: false }, // React already escapes
57     returnNull: false,
58     // Negotiation order (I18N_PLAN §2.3): persisted pref -> ?lang= -> browser -> en.
59     detection: {
60       order: ["localStorage", "querystring", "navigator"],
61       lookupLocalStorage: LANG_STORAGE_KEY,
62       lookupQuerystring: "lang",
63       caches: ["localStorage"],
64     },
65     react: { useSuspense: false },
66   });
67
68 // Keep <html lang> in sync with the active locale (screen readers + SEO use it).
Reflects the
69 // resolved content language, so an `en` fallback for an absent locale still reads as
`en`.
70 function syncDocumentLang(lng: string) {
71   if (typeof document !== "undefined") {
72     document.documentElement.lang = (i18n.resolvedLanguage || lng ||
DEFAULT_LOCALE).split("-")[0];
73   }
74 }
75 i18n.on("languageChanged", syncDocumentLang);
76 syncDocumentLang(i18n.language ?? DEFAULT_LOCALE);
77
78 export default i18n;
79

```

21. assistant (2026-06-21T19:32:53.235Z)

`i18n/index.ts` has real runtime logic (`syncDocumentLang` with branches) and is correctly included (not excluded), so it counts. It's exercised by `i18n.test.tsx`. Good ? that confirms the exclude list is genuinely minimal and not hiding logic.

I now have everything I need to assess the four points. Let me synthesize.

Key findings on the threshold tightness/gaming question: The functions floor is 80 vs. measured 82.35 ? that's a 2.35-point slack. With function counts in a small SPA, dropping below 80 from 82.35 means a few uncovered functions. Let me quickly gauge the function-count magnitude to judge whether the 80 floor is meaningful or trivially gameable, by checking how many functions exist roughly. The slack analysis is the load-bearing part.

The current numbers: statements/lines 92.61 (floor 92, slack 0.61), branches 87.74 (floor 87, slack 0.74), functions 82.35 (floor 80, slack 2.35). All floors are below current, so CI is green. None is above current. Now I'll evaluate gaming vectors and write findings.

22. user (2026-06-21T19:33:23.827Z)

Structured output provided successfully

23. assistant (2026-06-21T19:33:33.291Z)

Review complete. The coverage config is sound and I returned structured findings via StructuredOutput.

Summary of the assessment (verdict: ship):

1. Excludes verified by opening each file ? all four are genuinely non-logic: `web/src/main.tsx` (bootstrap), `web/src/api/types.ts` (interfaces only, would be uncoverable if included), `web/src/test/setup.ts` (test-harness shim), `web/src/vite-env.d.ts` (type augmentation). Files that look config-ish but carry real logic ? `web/src/i18n/index.ts` and `web/src/lib/log.ts` ? are correctly NOT excluded and still count.

2. `all: true` + `include: src/\*\*/\*.ts,tsx` is correct ? untested new files are counted as 0% and drag the ratio down, not silently ignored. No competing `vitest.config.\*` and no package.json override, so the block is authoritative.

3. Thresholds all sit just below measured (92/92 vs 92.61, 87 vs 87.74, 80 vs 82.35) ? green now, none above current, ratchet direction correct. Statements/lines/branches are tight (sub-1-point slack).

4. Two residual gaming vectors flagged as minor: the functions floor (80) has 2.35 pts of slack (loosest gate), and there's no `perFile` so a small untested module can ride in below the global aggregate. Both acceptable for a first floor; the obvious dodges (excluding source, dropping a test) are closed.